

DAY 5 — SESSION 2: DECODERS & ENCODERS

BLOCK 1: CONCEPT (10–12 min)

Part A — Decoders

A **decoder** takes a binary code as input and **activates exactly one output line**.

Think of it as: "which one of N things does this number refer to?"

Real hardware use cases:

- **Memory addressing** — a 3-bit address decodes to select one of 8 memory banks
 - **Instruction decode** — CPU sees opcode 101, decoder activates the correct execution unit
 - **Register file** — 3-bit register number selects one of 8 registers
-

2-to-4 Decoder — Truth Table

2 input bits → 4 output lines. Exactly one output is HIGH at any time.

in[1] in[0] out[3] out[2] out[1] out[0]

0 0 0 0 0 1

0 1 0 0 1 0

1 0 0 1 0 0

1 1 1 0 0 0

Pattern: out[i] is HIGH when in == i. Only one HIGH at a time.

3-to-8 Decoder — Concept

Same idea, scaled up. 3 input bits → 8 output lines.

in = 000 → out = 00000001 (out[0] active)

in = 001 → out = 00000010 (out[1] active)

in = 010 → out = 00000100 (out[2] active)

in = 011 → out = 00001000 (out[3] active)

...

in = 111 → out = 10000000 (out[7] active)

This is called **one-hot encoding**. Exactly one bit is 1. This appears constantly in FSMs, arbiters, and instruction decoders.

The Enable Pin

Real decoders have an **enable** input. When enable is LOW, all outputs are forced LOW regardless of the input. This allows you to chain decoders to build larger ones.

enable=0 → out = 00000000 (disabled, everything off)

enable=1 → normal decode operation

Part B — Encoders

An **encoder** is the **opposite of a decoder**.

It takes N input lines (one-hot) and produces a binary code.

DECODER: binary → one-hot

ENCODER: one-hot → binary

Real use case: you have 8 interrupt lines. Multiple might be asserted. You need to know **which one to service**. The encoder converts the active line into a number.

Priority Encoder

What happens if **two inputs are active at the same time** in a regular encoder? Output is undefined — garbage.

A **priority encoder** solves this. If multiple inputs are active, the **highest-numbered active input wins**.

in[3]	in[2]	in[1]	in[0]	out[1]	out[0]	valid
-------	-------	-------	-------	--------	--------	-------

0	0	0	0	X	X	0
---	---	---	---	---	---	---

0	0	0	1	0	0	1
---	---	---	---	---	---	---

0	0	1	X	0	1	1
---	---	---	---	---	---	---

0	1	X	X	1	0	1
---	---	---	---	---	---	---

in[3] in[2] in[1] in[0] out[1] out[0] valid

1 X X X 1 1 1

X = don't care — that input doesn't matter because a higher one dominates.

valid = 0 when no input is active (output is meaningless).

Common Mistakes

Mistake 1: Decoder with missing default → LATCH

// WRONG

always @(*) begin

 case (in)

 2'b00: out = 4'b0001;

 2'b01: out = 4'b0010;

 // forgot 10 and 11 → LATCH

 endcase

end

Mistake 2: Encoder with no valid signal If all inputs are 0, output is 00 — same as when in[0] is active. Without valid, you cannot tell the difference. Always include a valid output on encoders.

Mistake 3: Regular encoder with multiple inputs active Output becomes OR of multiple codes — meaningless. Use a **priority encoder** whenever inputs can overlap.

BLOCK 2: CODE & PRACTICAL (10–12 min)

Design 1: 2-to-4 Decoder with Enable

// FILE: decoder2to4.v

// 2-to-4 Decoder with active-high enable

// Synthesizable RTL — Verilog-2001

module decoder2to4 (

 input wire [1:0] in, // 2-bit binary input

```

input wire    enable, // active-high enable

output reg [3:0] out    // 4-bit one-hot output
);

always @(*) begin
    if (!enable) begin
        // When disabled, all outputs go LOW
        out = 4'b0000;
    end else begin
        case (in)
            2'b00: out = 4'b0001; // out[0] active
            2'b01: out = 4'b0010; // out[1] active
            2'b10: out = 4'b0100; // out[2] active
            2'b11: out = 4'b1000; // out[3] active
            default: out = 4'b0000;
        endcase
    end
end

endmodule

```

Why if (!enable) first?

Enable takes priority over the decode logic. This is the correct hardware behavior — enable gates the entire decoder output. Handle the override condition first, then normal operation.

Design 2: 3-to-8 Decoder

```

// FILE: decoder3to8.v

// 3-to-8 Decoder with enable

// Notice: exact same pattern, just scaled up

```

```

module decoder3to8 (
    input wire [2:0] in,    // 3-bit binary input
    input wire    enable, // active-high enable
    output reg [7:0] out    // 8-bit one-hot output
);
always @(*) begin
    if (!enable) begin
        out = 8'h00;    // 8'h00 = 8'b00000000, cleaner to write
    end else begin
        case (in)
            3'b000: out = 8'b00000001;
            3'b001: out = 8'b00000010;
            3'b010: out = 8'b00000100;
            3'b011: out = 8'b00001000;
            3'b100: out = 8'b00010000;
            3'b101: out = 8'b00100000;
            3'b110: out = 8'b01000000;
            3'b111: out = 8'b10000000;
            default: out = 8'h00;
        endcase
    end
end

endmodule

```

Spot the pattern:

Each output is just $1 \ll \text{in}$. Output[0] = 00000001, Output[1] = 00000010, Output[2] = 00000100. Each step shifts the 1 left by one position. That is exactly what a decoder does in hardware — it maps a binary number to a shift amount.

Design 3: 4-to-2 Priority Encoder

```
// FILE: priority_encoder4to2.v

// 4-input Priority Encoder
// Highest active input wins
// valid=0 when no input is active

module priority_encoder4to2 (
    input wire [3:0] in, // 4 request lines
    output reg [1:0] out, // 2-bit encoded output
    output reg valid // 1 = at least one input active
);

always @(*) begin
    // Default assignments — prevents latch
    // If nothing matches, these values hold
    out = 2'b00;
    valid = 1'b0;

    // Priority: in[3] > in[2] > in[1] > in[0]
    // if-else chain enforces priority naturally
    // First true condition wins, rest are ignored
    if (in[3]) begin
        out = 2'b11;
        valid = 1'b1;
    end else if (in[2]) begin
        out = 2'b10;
        valid = 1'b1;
    end else if (in[1]) begin
        out = 2'b01;
```

```

        valid = 1'b1;
    end else if (in[0]) begin
        out  = 2'b00;
        valid = 1'b1;
    end

    // If none active: out=00, valid=0 (defaults hold)

end

```

```
endmodule
```

Why if-else and not case here?

case treats all branches as equal priority — first match wins only because of simulation ordering, not explicit priority. if-else if chain is the correct RTL pattern for priority logic because the **hardware it implies is a priority chain** — each condition blocks the ones below it. Synthesizers understand this pattern and generate correct priority logic.

Why set defaults at the top?

```

out  = 2'b00;

valid = 1'b0;

```

This is a critical RTL habit. Setting defaults at the top of an always @(*) block means every output gets assigned **before** the if-else chain runs. Even if no branch is taken, the output has a value. This **prevents latches**. Always do this in combinational always blocks.

Testbench — Priority Encoder

```
// FILE: tb_priority_encoder.v
```

```
`timescale 1ns/1ps
```

```
module tb_priority_encoder;
```

```
    reg [3:0] in;
```

```

wire [1:0] out;

wire    valid;

priority_encoder4to2 dut (
    .in(in), .out(out), .valid(valid)
);

integer pass_count, fail_count;

// Check output AND valid signal together
task check;
    input [1:0] exp_out;
    input    exp_valid;
    input [63:0] test_num;
    begin
        #1;
        if (out === exp_out && valid === exp_valid) begin
            $display("PASS [Test %0d] in=%b out=%0d valid=%b",
                test_num, in, out, valid);
            pass_count = pass_count + 1;
        end else begin
            $display("FAIL [Test %0d] in=%b out=%0d(exp %0d) valid=%b(exp %b)",
                test_num, in, out, exp_out, valid, exp_valid);
            fail_count = fail_count + 1;
        end
    end
endtask

```



```
initial begin
```

```
    pass_count = 0;
```

```
    fail_count = 0;
```

```
    // No inputs active
```

```
    in = 4'b0000; check(2'b00, 1'b0, 1);
```

```
    // Single inputs
```

```
    in = 4'b0001; check(2'b00, 1'b1, 2); // only in[0]
```

```
    in = 4'b0010; check(2'b01, 1'b1, 3); // only in[1]
```

```
    in = 4'b0100; check(2'b10, 1'b1, 4); // only in[2]
```

```
    in = 4'b1000; check(2'b11, 1'b1, 5); // only in[3]
```

```
    // Priority tests — multiple inputs active
```

```
    in = 4'b0011; check(2'b01, 1'b1, 6); // in[1] beats in[0]
```

```
    in = 4'b0111; check(2'b10, 1'b1, 7); // in[2] beats in[1],in[0]
```

```
    in = 4'b1111; check(2'b11, 1'b1, 8); // in[3] beats all
```

```
    in = 4'b1010; check(2'b11, 1'b1, 9); // in[3] beats in[1]
```

```
    $display("-----");
```

```
    $display("TOTAL: %0d passed, %0d failed", pass_count, fail_count);
```

```
    if (fail_count == 0)
```

```
        $display("ALL TESTS PASSED");
```

```
    else
```

```
        $display("FAILURE DETECTED");
```

```
    $finish;
```

```
end
```

```
initial begin

    $dumpfile("priority_encoder.vcd");

    $dumpvars(0, tb_priority_encoder);

end
```

```
endmodule
```

Notice the task now checks two outputs:

```
if (out === exp_out && valid === exp_valid)
```

Both must be correct for the test to pass. This is important — valid is part of the contract of this module. Checking only out would miss bugs in valid.

Run It

```
iverilog -o sim/priority_enc sim/priority_encoder4to2.v tb/tb_priority_encoder.v
vvp sim/priority_enc
```

BLOCK 3: MINI TASK (5–8 min)

Your Task

Write a **3-to-8 decoder testbench** for decoder3to8.v.

Requirements:

1. Test **all 8 input combinations** (000 through 111) with enable = 1
2. Test **at least 2 cases** with enable = 0 — output must be 8'h00
3. Self-checking — PASS/FAIL for every test
4. Summary at the end

One thinking question before you write:

When enable = 0 and in = 3'b101, what should out be?

Answer that first, then write the testbench. Post your code when ready.

You are right. I skipped it.

DEMUX belongs in this session alongside decoder. They are related circuits and you need both before moving to the ALU.

DEMULTIPLEXER — FULL LESSON

CONCEPT

A **demultiplexer (DEMUX)** is the opposite of a MUX.

MUX: many inputs → 1 output (selector picks which INPUT to read)

DEMUX: 1 input → many outputs (selector picks which OUTPUT to write)

One data input comes in. The select lines decide which output lane it gets routed to. All other outputs are 0.

Real Hardware Use Cases

- **Data routing** — one serial data stream gets directed to one of several processing units
 - **Bus demultiplexing** — one shared bus routes data to individual registers
 - **Communication systems** — one received signal gets routed to the correct receiver
-

1-to-4 DEMUX — Truth Table

sel[1] sel[0] out[0] out[1] out[2] out[3]

0	0	in	0	0	0
0	1	0	in	0	0
1	0	0	0	in	0
1	1	0	0	0	in

The selected output carries the input value. Everything else is 0.

DEMUX vs Decoder — The Key Difference

DECODER: output[sel] = 1, all others = 0

DEMUX: output[sel] = in, all others = 0

A decoder is a DEMUX where the input is permanently tied to 1. That is the only difference. This is why they are taught together.

CODE

1-to-2 DEMUX

```
// FILE: demux1to2.v
// 1-to-2 Demultiplexer
// Parameterized data width

module demux1to2 #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] in, // single data input
    input wire sel, // select line
    output reg [WIDTH-1:0] out0, // output lane 0
    output reg [WIDTH-1:0] out1 // output lane 1
);

always @(*) begin
    // Default both outputs to 0
    // Critical: prevents latch on unselected output
    out0 = {WIDTH{1'b0}};
    out1 = {WIDTH{1'b0}};

    case (sel)
        1'b0: out0 = in; // route input to lane 0
        1'b1: out1 = in; // route input to lane 1
    endcase
end
```

```
end
```

```
endmodule
```

Why defaults at the top here?

At any moment, only ONE output is active. The other must be 0. If you only assign inside the case without defaults, the unselected output has no driver — that is a latch. The defaults ensure every output gets a value every time the block runs.

1-to-4 DEMUX

```
// FILE: demux1to4.v
```

```
// 1-to-4 Demultiplexer
```

```
module demux1to4 #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] in,
    input wire [1:0] sel,
    output reg [WIDTH-1:0] out0,
    output reg [WIDTH-1:0] out1,
    output reg [WIDTH-1:0] out2,
    output reg [WIDTH-1:0] out3
);
    always @(*) begin
        // Default all outputs to 0
        out0 = {WIDTH{1'b0}};
        out1 = {WIDTH{1'b0}};
        out2 = {WIDTH{1'b0}};
        out3 = {WIDTH{1'b0}};
```

```

        case (sel)
            2'b00: out0 = in;
            2'b01: out1 = in;
            2'b10: out2 = in;
            2'b11: out3 = in;
            default: ; // all outputs already 0 from defaults above
        endcase
    end

endmodule

```

Testbench — 1-to-4 DEMUX

```
// FILE: tb_demux1to4.v
```

```
`timescale 1ns/1ps
```

```
module tb_demux1to4;
```

```
    parameter WIDTH = 8;
```

```
    reg [WIDTH-1:0] in;
```

```
    reg [1:0]      sel;
```

```
    wire [WIDTH-1:0] out0, out1, out2, out3;
```

```
    demux1to4 #(.WIDTH(WIDTH)) dut (
```

```
        .in(in),
```

```
        .sel(sel),
```

```
        .out0(out0), .out1(out1),
```

```
.out2(out2), .out3(out3)
);
```

```
integer pass_count, fail_count;
```

```
// Check all 4 outputs at once
```

```
task check;
```

```
input [WIDTH-1:0] exp0, exp1, exp2, exp3;
```

```
input [63:0] test_num;
```

```
begin
```

```
#1;
```

```
if (out0===exp0 && out1===exp1 && out2===exp2 && out3===exp3) begin
```

```
$display("PASS [Test %0d] sel=%b in=%0d | out=%0d,%0d,%0d,%0d",
```

```
test_num, sel, in, out0, out1, out2, out3);
```

```
pass_count = pass_count + 1;
```

```
end else begin
```

```
$display("FAIL [Test %0d] sel=%b in=%0d | got=%0d,%0d,%0d,%0d  
exp=%0d,%0d,%0d,%0d",
```

```
test_num, sel, in, out0,out1,out2,out3, exp0,exp1,exp2,exp3);
```

```
fail_count = fail_count + 1;
```

```
end
```

```
end
```

```
endtask
```

```
initial begin
```

```
pass_count = 0;
```

```
fail_count = 0;
```

```

in = 8'd42;

// Route to each output — only selected output gets 'in', rest are 0
sel = 2'b00; check(8'd42, 8'd0, 8'd0, 8'd0, 1);
sel = 2'b01; check(8'd0, 8'd42, 8'd0, 8'd0, 2);
sel = 2'b10; check(8'd0, 8'd0, 8'd42, 8'd0, 3);
sel = 2'b11; check(8'd0, 8'd0, 8'd0, 8'd42, 4);

// Change input value, re-verify routing works
in = 8'd255;
sel = 2'b00; check(8'd255, 8'd0, 8'd0, 8'd0, 5);
sel = 2'b11; check(8'd0, 8'd0, 8'd0, 8'd255, 6);

// Zero input — all outputs should be 0 regardless of sel
in = 8'd0;
sel = 2'b10; check(8'd0, 8'd0, 8'd0, 8'd0, 7);

$display("-----");
$display("TOTAL: %0d passed, %0d failed", pass_count, fail_count);
if (fail_count == 0)
    $display("ALL TESTS PASSED");
else
    $display("FAILURE DETECTED");

$finish;

end

initial begin

```



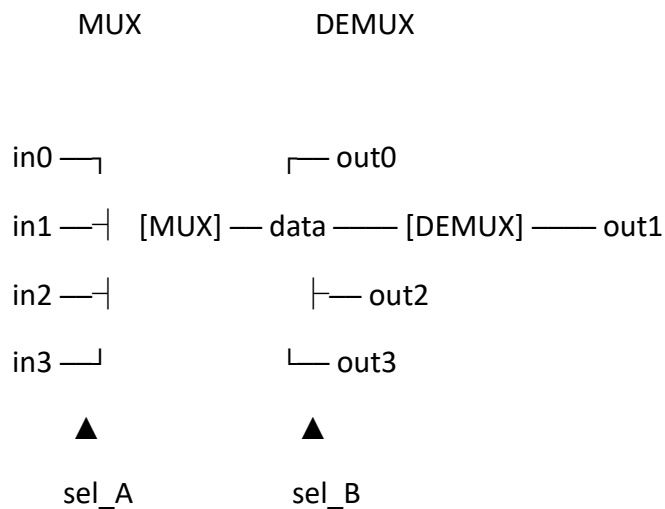
```

    $dumpfile("demux1to4.vcd");
    $dumpvars(0, tb_demux1to4);
end

```

```
endmodule
```

MUX / DEMUX Together — The Full Picture



MUX picks which source. DEMUX picks which destination. Together they form a **data routing fabric** — the backbone of bus systems and interconnects.

MINI TASK

Write the **1-to-2 DEMUX testbench** for demux1to2.v.

Requirements:

1. Test both sel values with a non-zero input
2. Verify the unselected output is always 0
3. Test with boundary input values (0 and 255)
4. Self-checking PASS/FAIL

Post your code. Then we continue to the decoder mini task from before — you still owe that one too.